Roll no: 37

Practical 1a: Floyd's Triangle till N Rows in C#

```
using System;
namespace pracla
{
    internal class Program
    {
        static void Main(string[] args)

        {
            Console.WriteLine("Floyds triangle");
            Console.Write("Enter the number of rows:");
            int n = int.Parse(Console.ReadLine());
            int num = 1;
            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(num + " ");
                    num++;
                }
                Console.WriteLine();
            }Console.ReadKey();

        }}}
```

```
Floyds triangle
Enter the number of rows:5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Practical 1b: Arrange Names of Cities in Sorted Order

```
using System;
namespace SortCities
{
    class Program
    {
        static void Main(string[] args)
        {
    string[] cities = new string[10];

       Console.WriteLine("=== City Name Sorter ===");
        Console.WriteLine("Enter names of 10 cities:");
            for (int i = 0; i < 10; i++)
            {
                cities[i] = Console.ReadLine();
            }

            // Sort array using Array.Sort()
Array.Sort(cities);
Console.WriteLine("\n--- Cities in Sorted Order ---");
            foreach(string i in cities)
            {
                Console.WriteLine(i);
            }}}}
```

```
=== City Name Sorter ===
Enter names of 10 cities:
delhi
chennai
surat
jaipur
mumbai
pune
bangalore
hyderabad
kolkata
ahmedabad

--- Cities in Sorted Order ---
ahmedabad
bangalore
chennai
delhi
hyderabad
jaipur
kolkata
mumbai
pune
surat
```

9/7/26

Practical no: 02

2. //Function Overloading rollno: 77

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace B4_Arjun_Nitin
{
    class area
    {
        public void CalculateArea(int side)
        {
            Console.WriteLine("Area of Square = " + (side * side));
        }
        public void CalculateArea(int length,int breadth)
        {
            Console.WriteLine("Area of Rectangle =" +(breadth * length));
        }
        public double CalculateArea(float p,double rad)
        {
           return(p*rad*rad);
        }
    }
    internal class functionoverloading
    {
        static void Main(string[] args)
        {
            area obj = new area();
            Console.WriteLine("Enter the side");
            int side = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(side);

            Console.WriteLine("Enter the length of Rectangle :");
            int length = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Enter the breadth of Rectangle :");
            int breadth = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(length, breadth);


            Console.WriteLine("Enter the Radius of circle :");
            double rad =double.Parse(Console.ReadLine());
            Console.WriteLine("circle"+obj.CalculateArea(3.14f, rad));
        }

    }
}
```

```
Enter the side
2
Area of Square = 4
Enter the length of Rectangle :
3
Enter the breadth of Rectangle :
2
Area of Rectangle =6
Enter the Radius of circle :
5
circle78.5000026226044
```

18/7/26

B //Inheritance Rollno- 77

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Runtime.ConstrainedExecution;

using System.Threading.Tasks;

namespace B4_Arjun_Nitin

{
    class vehicle
    {
        string name = "Audi";
        public void dispvehicle()
        {
            Console.WriteLine("vehicle name =" + name);
        }
    }
    class car : vehicle
    {
        string model = "2026";
        public void dispcar()
        {
            dispvehicle();
            Console.WriteLine("Car model =" + model);
        }
    }
    class sportscar : car
    {
        int speed=450;
        public void dispsportscar()
        {
            dispcar();
            Console.WriteLine("Max speed =" + speed);
        }
    }
    class bike : vehicle
    {
        string type = "classic";
        public void dispbike()
        {

            dispvehicle();
            Console.WriteLine("bike type =" + type);
        }
    }

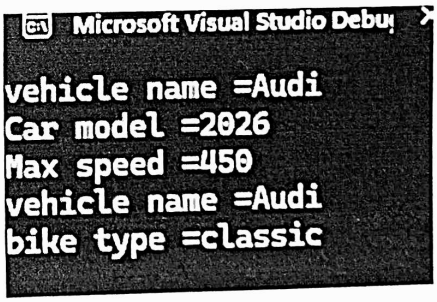
    internal class inheritance
    {
        static void Main(string[] args)
        {

            sportscar s = new sportscar();
            bike b = new bike();

           s.dispsportscar();
            b.dispbike();



        } }
}
```



18/7/26

```
//Interface Code rollno=

using System;

interface Area
{
    double compute(double x);
}
class square : Area
{
   public  double compute(double x)
    {
        return x*x;
    }
}
class circle : Area
{
public double compute(double x)
    {
        return ( Math.PI* x * x);
    }
}
class interf
{
    public static void Main(string[] args)
    {
        square sq=new square();
        circle c=new circle();
        Area a;
        a = sq as Area;
        Console.WriteLine("Area of Square=" + a.compute(10.0));
        a=c as Area;
        Console.WriteLine("Area of Circle=" + a.compute(10.0));

    }
}
```

Output :

Area of Square=100
Area of Circle=314.159265358979

31/7/24

Exception Handling

```
using System;
class invalidage : Exception
{
    public  invalidage(String msg):base(msg)
    {

    }
}
class exceptHan
{
    static void ValidateAge(int age)
    {
        if (age < 0 || age > 100)
        {
            throw new invalidage("invalid Age");
        }
    }

        public static void Main(string[] args)
    {
        try
        {
            int a = 10, b = 2;
            Console.WriteLine(a / b);

        }
        catch(DivideByZeroException) {
            Console.Write("Deniminator should not be zero");

        }
        finally
        {
            Console.Write("In Finally....");

        }
        try
        {
            Console.Write("\nEnter age:");
            int age = int.Parse(Console.ReadLine());
            ValidateAge(age);
        }
        catch(invalidage ex)
        {
            Console.WriteLine("Try with other Input !");

        }

        }
    }
```

31/7/26

Ouput:

```
5
In Finally....
Enter age:35
```

```
5
In Finally....
Enter age:109
Try with other Input !
```

Practical 3a: Arithmetic Methods via C# Delegates

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace caldelegate
{
    delegate double ArithmeticOperation(double a, double b);

    class Calculator
    {
      public static double Add(double a, double b)
      {
        return a + b;
      }
      public static double Subtract(double a, double b)
      {
        return a - b;
      }
      public static double Multiply(double a, double b)
      {
        return a * b;
      }
      public static double Divide(double a, double b)
      {
        if (b == 0)
        {
          Console.WriteLine("Division by zero is not possible.");
          return 0;
        }
        return a / b;
    }

    static void Main(string[] args)
    {
      Console.Write("Enter First Number: ");
      double num1 = Convert.ToDouble(Console.ReadLine());

      Console.Write("Enter Second Number: ");
      double num2 = Convert.ToDouble(Console.ReadLine());
      ArithmeticOperation op;

      op = Add;
      Console.WriteLine("Addition = " + op(num1, num2));
```

```
Enter First Number: 300
Enter Second Number: 200
Addition = 500
Subtraction = 100
Multiplication = 60000
Division = 1.5
```

```
        op = Subtract;
        Console.WriteLine("Subtraction = " + op(num1, num2));

        op = Multiply;
        Console.WriteLine("Multiplication = " + op(num1, num2));

        op = Divide;
        Console.WriteLine("Division = " + op(num1, num2));
    }}}
```

Practical 3b: Multicast Delegate

```
using System;

    namespace multicast_delegate
    {

        delegate void Mdelegate();


        class DM
        {
            static public void display()
        {
            Console.WriteLine("hello");
        }
        static public void Print()
        {
            Console.WriteLine("hello world");
        }

     }
     class Mtest
     {
         public static void Main()
         {
             Mdelegate m1 = new Mdelegate(DM.display);
             Mdelegate m2 = new Mdelegate(DM.Print);
             Mdelegate m3 = m1 + m2;
             Mdelegate m4 = m2 + m1;
             Mdelegate m5 = m3 - m2;
             Console.WriteLine("delegate M3:");
           m3(); Console.WriteLine("delegate M4:");
             m4(); Console.WriteLine("delegate M5:");
             m5();}}}
```

```
Multicast delegate
delegate M3:
hello
hello world
delegate M4:
hello world
hello
delegate M5:
hello
```

Practical 3c: BankAccount Class with Property Validation

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace bankaccount
{
class BankAccount
    {       // Private field for accountnumber
```

```
        private string accountnumber;
//Property for AccountNumber

        public string AccountNumber { get; set; }

        // Private field for balance
        private double balance;
// Property with validation
        public double Balance
        {
            get { return balance; }
            set
            {
                if (value >= 0)
                {
                    balance = value;
                }
                else
                {
                    Console.WriteLine("Balance cannot be negative.");
                }
            }
        }

// Constructor
        public BankAccount(string accountNumber, double balance)
        {
            AccountNumber = accountNumber;
            Balance = balance;
        }
// Method to display account details
        public void Display()
        {
            Console.WriteLine("\nAccount Details");
            Console.WriteLine("Account Number: " + AccountNumber);
            Console.WriteLine("Balance: " + Balance);
        }
    }

    class Program
    {
        static void Main()
        {
            Console.Write("Enter Account Number: ");
            string accNo = Console.ReadLine();

            Console.Write("Enter Balance: ");
            double bal = Convert.ToDouble(Console.ReadLine());

            BankAccount account = new BankAccount(accNo, bal);

            account.Display();

        }
        Console.Readkey();}}
```

31/71

4a: Student Registration Form

STEPS

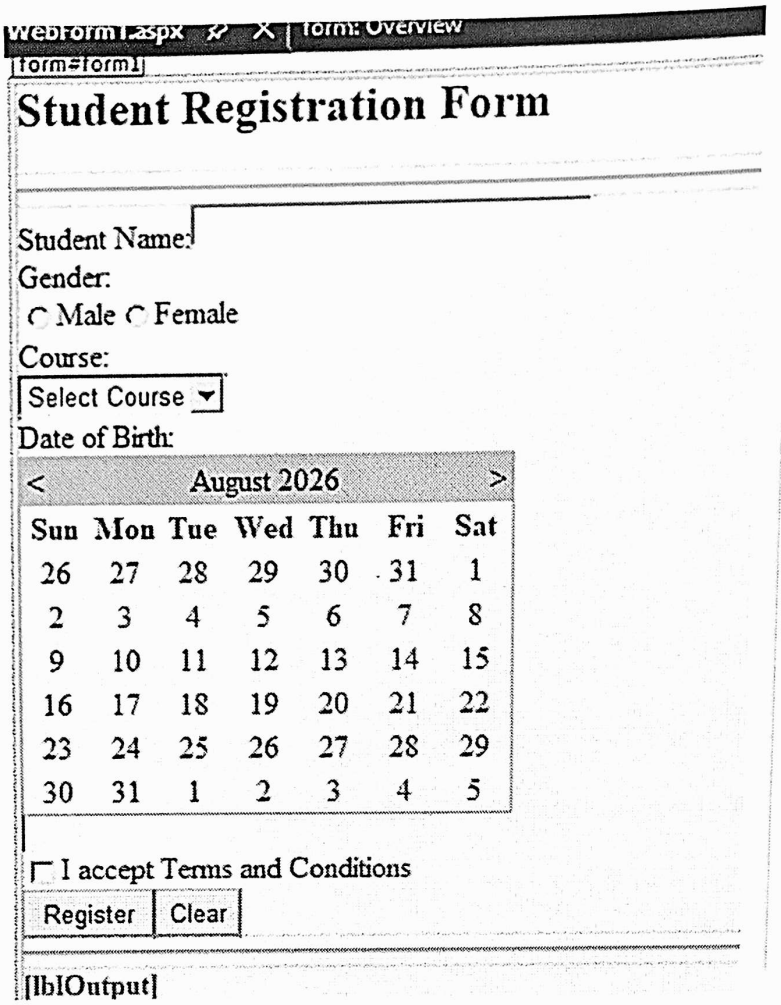
1. Open Visual Studio → File → New → Project → ASP.NET Web Application (.NET Framework) →Click Next.
2. Project name→ Click Create
3. Choose Empty →Add folders &core reference choose webform →Click Create
4. Add Empty Webform →Solution Explorer:Right-click Project Name →Add →Web form
5. Enter WebForm name →Click Add.
6. Open WebForm1.aspx and add the following markup
7. Open WebForm1aspx.cs and add the code-behind logic

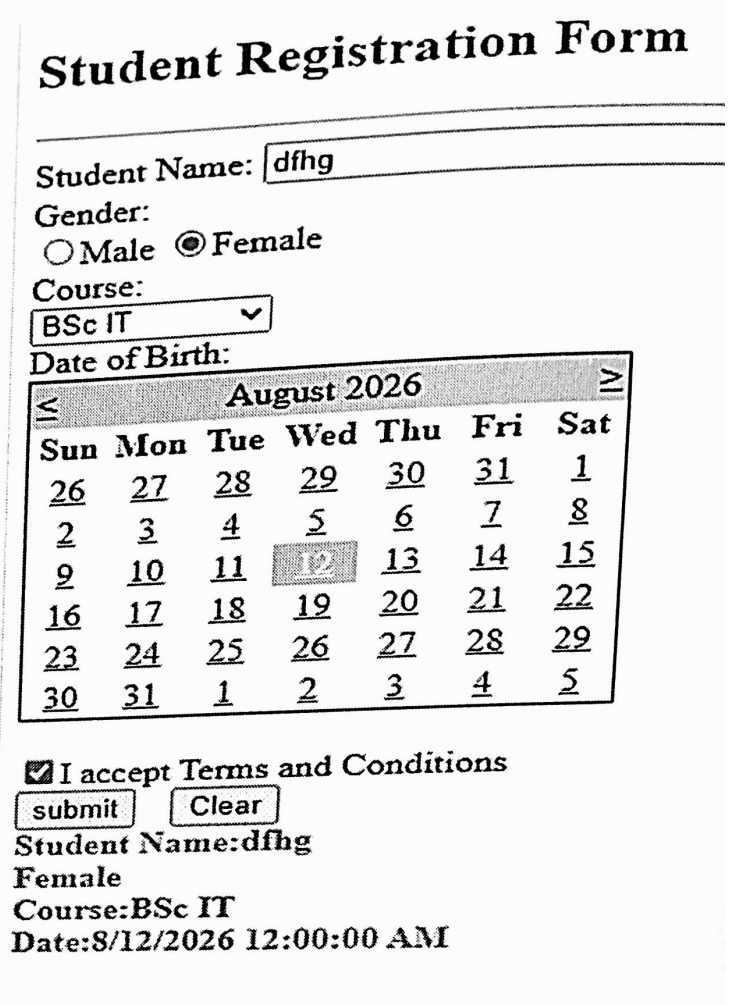
WebForm1.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs"
Inherits="WebApplication1.WebForm1" %>
<!DOCTYPE html>
<form id="form1" runat="server">
<div><h2>Student Registration Form</h2> <hr /></div>
<asp:Label Text="Student Name:" runat="server" />
<asp:TextBox ID="txtName" runat="server" Width="200px" /> <br />
<asp:Label Text="Gender:" runat="server" /><br />
<asp:RadioButton ID="rbMale" GroupName="gender" Text="Male" runat="server" />
<asp:RadioButton ID="rbFemale" GroupName="gender" Text="Female" runat="server" /><br />
<asp:Label Text="Course:" runat="server" /> <br />
<asp:DropDownList ID="ddlCourse" runat="server"> <asp:ListItem Text="Select Course"
Value="" />
<asp:ListItem Text="BSc IT" Value="BSc IT" />
<asp:ListItem Text="BSc CS" Value="BSc CS" />
<asp:ListItem Text="BCA" Value="BCA" /> </asp:DropDownList> <br />
<asp:Label Text="Date of Birth:" runat="server" /> <br /><asp:Calendar ID="calDOB"
runat="server" /><br />
<asp:CheckBox ID="chkTerms" Text="I accept Terms and Conditions" runat="server" /><br />
<asp:Button ID="btnSubmit" runat="server" OnClick="btnSubmit_Click" Text="submit" />
  
<br /><asp:Label ID="lblOutput" runat="server" ForeColor="Green" Font-Bold="true" /><br
/></form></body></html>
```

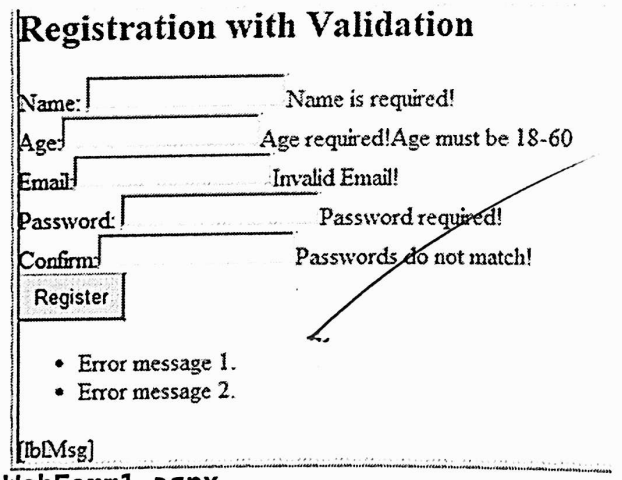
WebForm1.aspx.cs

```
using System;
namespace WebApplication1
{
public partial class WebForm1 : System.Web.UI.Page
{ protected void Page_Load(object sender, EventArgs e)
{}
protected void btnSubmit_Click(object sender, EventArgs e)
{
string gender = "";
if(rbMale.Checked)
{
gender = rbMale.Text;
}
else if(rbFemale.Checked)
{  gender = rbFemale.Text;
}
if (chkTerms.Checked)
{
lblOutput.Text = "Student Name:" + txtName.Text + "<br/>" + gender + "<br/>Course:" +
ddlCourse.SelectedValue + "<br/> Date:" + calDOB.SelectedDate.ToString();
}
else { lblOutput.Text = "please accept terms & condition";}}}}
```





4b: Registration Form with Validation Controls



7/8/26

WebForm1.aspx

```
<form id="form1" runat="server">
<h2>Registration with Validation</h2>
Name:
<asp:TextBox ID="txtName" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtName" ErrorMessage="Name is required!"
ForeColor="Red" runat="server" />
<br />
```

```
Age:<asp:TextBox ID="txtAge" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtAge" ErrorMessage="Age required!"
ForeColor="Red" runat="server" />
<asp:RangeValidator ControlToValidate="txtAge" Type="Integer" MinimumValue="18"
MaximumValue="60" ErrorMessage="Age must be 18-60" ForeColor="Red" runat="server" />
<br />
Email:<asp:TextBox ID="txtEmail" runat="server" />
<asp:RegularExpressionValidator ControlToValidate="txtEmail" ValidationExpression="^[\w-
\.]+@([\w-]+\.)+[\w-]{2,4}$" ErrorMessage="Invalid Email!" ForeColor="Red"
runat="server" />
<br />
Password:
<asp:TextBox ID="txtPass" TextMode="Password" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtPass" ErrorMessage="Password
required!" ForeColor="Red" runat="server" />
<br />
Confirm:<asp:TextBox ID="txtConfirm" TextMode="Password" runat="server" />
<asp:CompareValidator ControlToValidate="txtConfirm" ControlToCompare="txtPass"
ErrorMessage="Passwords do not match!" ForeColor="Red" runat="server" />
<br />
<asp:Button ID="btnReg" Text="Register" runat="server" OnClick="btnReg_Click"
Height="31px" />
<asp:ValidationSummary runat="server" ForeColor="Red" /> <asp:Label ID="lblMsg"
runat="server" ForeColor="Green" />
</form> </body> </html>
```

WebForm1.aspx.cs

```
protected void btnReg_Click(object sender, EventArgs e)
{
if (Page.IsValid)
{
lblMsg.Text = "Registration successful.";
}}
```

Web.config

```
<configuration>
<appSettings>
<add key="ValidationSettings:UnobtrusiveValidationMode" value="None" />
</appSettings>
<system.web>
```

Registration with Validation

Name: Name is required!
Age: 8 Age must be 18-60
Email: iko Invalid Email!
Password: .....
Confirm: ..... Passwords do not match!
Register

- Name is required!
- Age must be 18-60
- Invalid Email!
- Passwords do not match!

Registration with Validat

Name: abc
Age: 34
Email: abc@gmail.com
Password:
Confirm:
Register Registration successful.

7/8/26

4c: AdRotator Control

XMLFile.xml

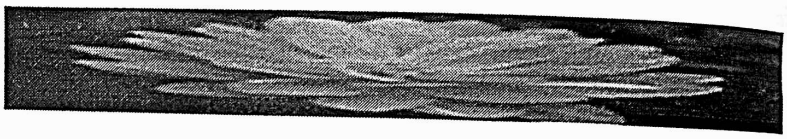
```
<?xmlversion="1.0"encoding="utf-8" ?>
<Advertisements>
<Ad>
<ImageUrl>image1.jpg</ImageUrl>
<NavigateUrl>Default3.aspx</NavigateUrl>
<AlternateText>advertisement</AlternateText>
<Impressions>1</Impressions>
<Keyword>food</Keyword>
</Ad>
<Ad>
<ImageUrl>image2.jpg</ImageUrl>
<NavigateUrl>Default.aspx</NavigateUrl>
<AlternateText>
advertisement
</AlternateText>
<Impressions>1</Impressions>
<Keyword>food</Keyword>
</Ad>
<Ad>
<ImageUrl>image3.jpg</ImageUrl>
<NavigateUrl>https://thebelgianwaffle.co/</NavigateUrl>
<AlternateText>
advertisement
</AlternateText>
<Impressions>1</Impressions>
<Keyword>food</Keyword>
</Ad>
<Ad>
<ImageUrl>image4.jpg</ImageUrl>
<NavigateUrl>https://mcdelivery.co.in/</NavigateUrl>
<AlternateText>
advertisement
</AlternateText>
<Impressions>1</Impressions>
<Keyword>coffee</Keyword>
</Ad>
<Ad>
<ImageUrl>image5.jpg</ImageUrl>
<NavigateUrl>https://www.zomato.com/mumbai/desi-chai-theka-4-vasai</NavigateUrl>
<AlternateText>
advertisement
</AlternateText>
<Impressions>1</Impressions>
<Keyword>coffee</Keyword>
</Ad>
</Advertisements>
```

Default.aspx

```
<form id="form1" runat="server">
<h2>AdRotator Control Demo</h2>
<p>Refresh the page to see a different advertisement:</p>
<div>
<asp:AdRotator ID="AdRotator1" runat="server" AdvertisementFile="~/ADS.xml"
Height="60px" Width="468px" />
<br />
<asp:Button ID="Button1" runat="server" Text="Show Another Ad" />
</div></form></body></html>
```

AdRotator Control Demo

Refresh the page to see a different advertisement:



Practical 5A: Create an MVC Application to perform basic Arithmetic Operations.

Steps

1: Open visual studio 2022/2026

2: Create new project

- Click on create a new project
- Search for MVC
- Select project type as ASP.NET Core Web App(Model-View-controller)
- Then click next

2.2 Name the project arithmeticmvc->Click Next->Select Framework (latest available)

3. **Create Model File**

- In solution Explorer (If this window is not seen, go to view Menu and select it.)
- Right-click: Models →Add→Class
- Enter File name as Calculator.cs and click Add

```
namespace arithmeticmvc.Models{
public class Calculator{
public double Number1{ get;set;}
public double Number2{ get;set;}
public required string operation { get; set; }
public double Result { get; set; }
```

Press Ctrl+ S to save

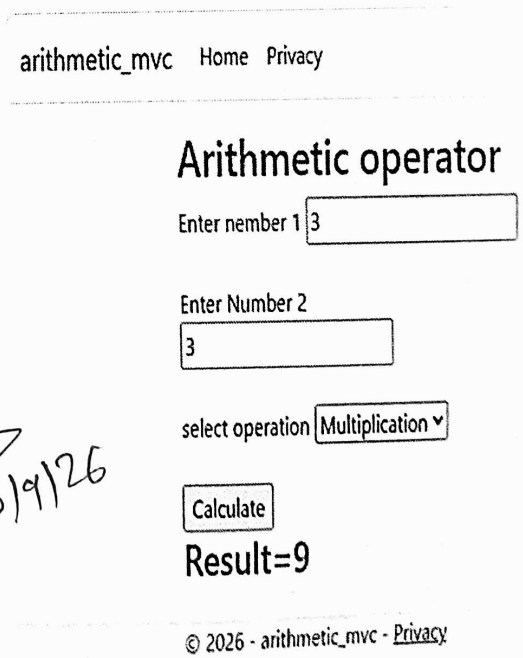
4. Open Controller

In solution explorer: open Controllers => Double click HomeController.cs

Modify Controller code:

Remove existing code inside HomeController and replace with:

```
using arithmeticmvc.Models;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;
namespace arithmeticmvc.Controllers{
public class HomeController : Controller
{ public IActionResult Index(Calculator calc){
switch (calc.operation){
case "+":
calc.Result = calc.Number1 + calc.Number2;
break;
case "-":
calc.Result = calc.Number1 - calc.Number2;
break;
case "*":
calc.Result = calc.Number1 * calc.Number2;
break;
case "/":
if (calc.Number2 != 0)
{
calc.Result = calc.Number1 / calc.Number2;
}
break;} return View(calc);}}}
```



8/9/26

Save the file

5. Open views Folder

In solution explorer: Open: => Views -> Home

Double-click :=> Index.cshtml

11: Replace Existing View Code: Delete all existing code and paste:

Index.cshtml

```
@model arithmeticmvc.Models.Calculator
<h2>
Arithmetic operator
</h2>
<form method="post">
<label>Enter nember 1</label>
<input type="number" step="any" asp-for="Number1" />
<br />  <br />
<label> Enter Number 2</label><br />
<input type="number" step="any" asp-for="Number2" /> <br /><br />
<label>select operation</label>
<select asp-for="operation">
<option value="+">Addition</option>
<option value="-">Subtraction</option>
<option value="*">Multiplication</option>
<option value="/">Division</option>
</select> <br /><br /><input type="submit" value="Calculate" /></form>
@if (Model != null)
{
<h3>
Result=@Model.Result
</h3>
}
```

5b: Different Types of Cookies

1. Create Project

Step 1: Open Visual Studio 2026
Step 2: Create → `ASP.NET Web Application (.NET Framework)
Step 3: Select: `Empty` and `Check Web Forms`
Step 4: Click Create

2. Add Web Form

Right-click Project → `Add` → `Web Form`
Name: `CookieDemo.aspx`
3.

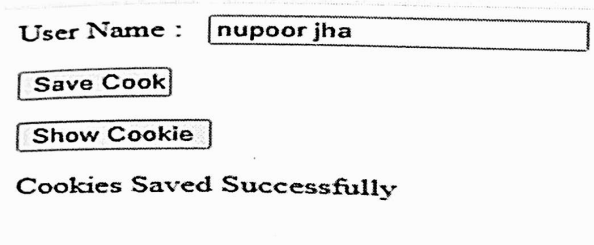
Add Controls Label Text	Control Type	ID
User Name	TextBox	txtName
Save Cookie	Button	btnSave
Show Cookies	Button	btnShow
Output	Label	lblOutput

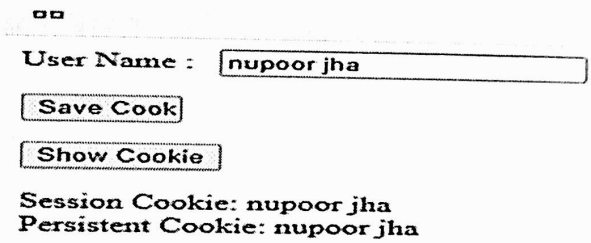
Write Code for Save Cookie Button

```
protected void btnSave_Click(object sender, EventArgs e) {
HttpCookie sessionCookie = new HttpCookie("SessionUser");
sessionCookie.Value = txtName.Text;
Response.Cookies.Add(sessionCookie);
HttpCookie persistentCookie = new HttpCookie("PersistentUser");
persistentCookie.Value = txtName.Text;
persistentCookie.Expires = DateTime.Now.AddDays(7);
Response.Cookies.Add(persistentCookie);
lblOutput.Text = "Cookies Saved Successfully";} }
```

Write Code for Show Cookie Button

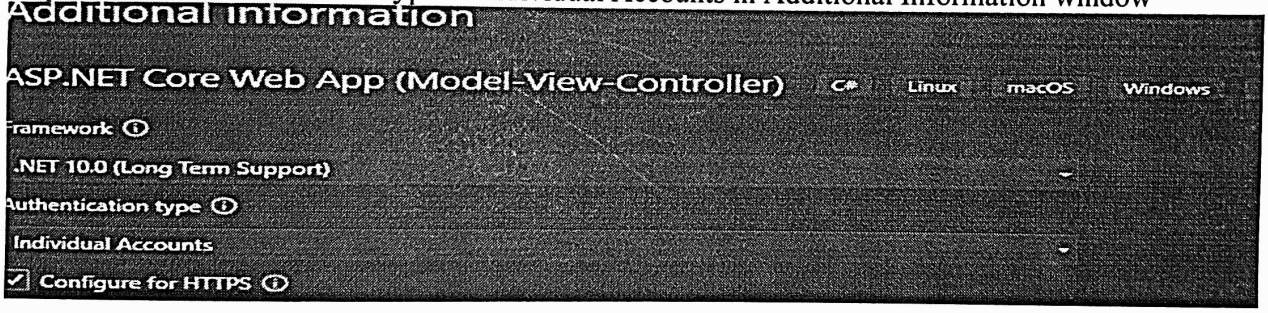
```
protected void btnShow_Click(object sender, EventArgs e)
{
string sessionData = Request.Cookies["SessionUser"].Value;
string persistentData = Request.Cookies["PersistentUser"].Value;
lblOutput.Text = "Session Cookie: " + sessionData +
"<br/>Persistent Cookie: " + persistentData;
}
```



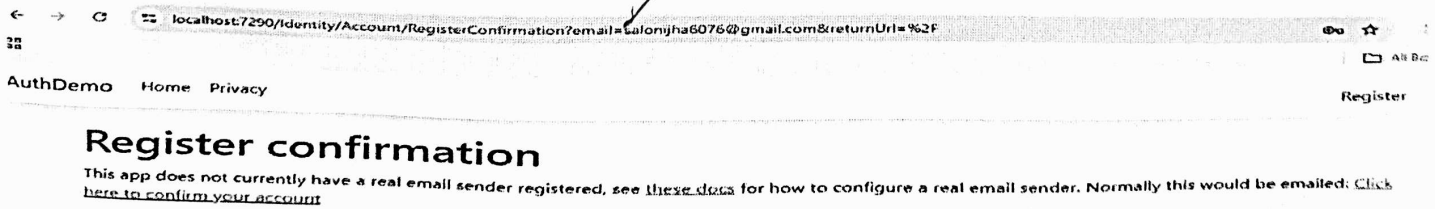


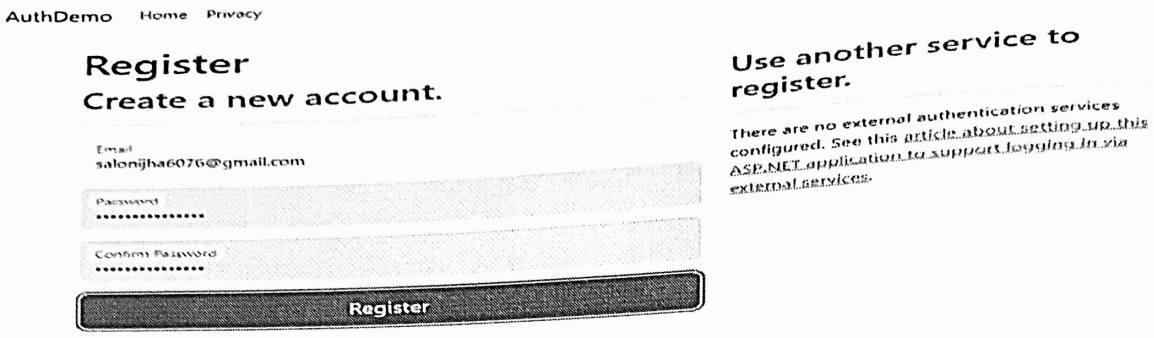
Practical 5D: Authentication and authorization in ASP.NET Core.

Step 1: Create MVC Project. Project (ASP.Net Core Web App (Model-View-Controller))
Name AuthDemo and Framework .NET 10.Click Next.
Step 2: Select Authentication Type as Individual Accounts in Additional Information window

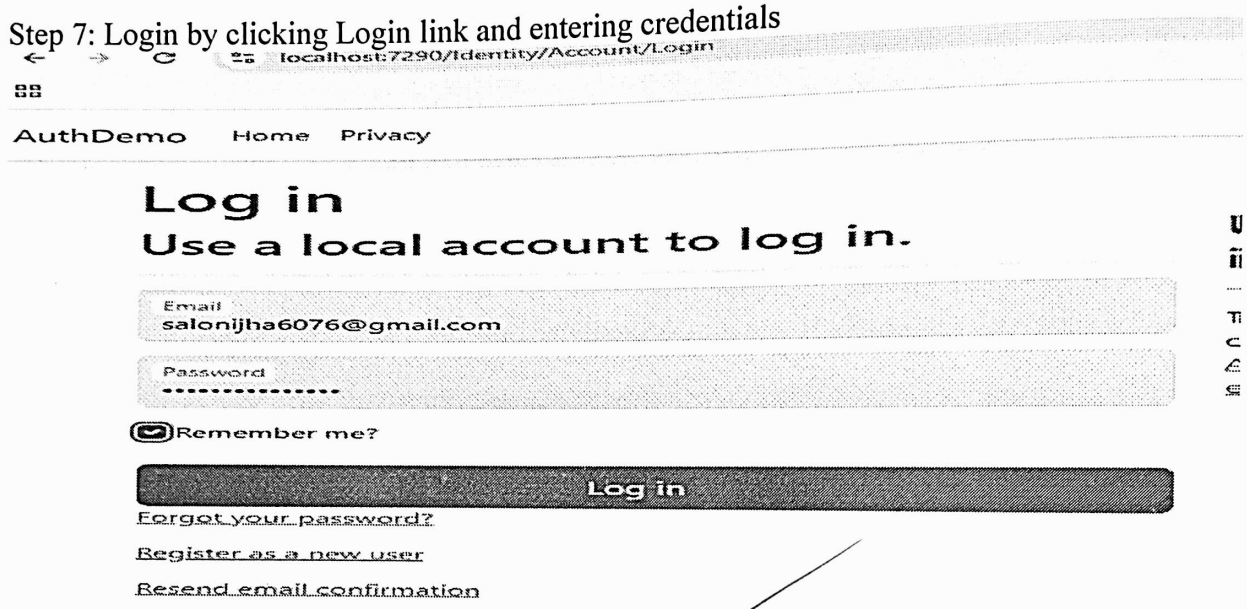


Step 3: Build Project using Rebuild Solution
Go to Build-> Rebulid Solution
Step 4: Create Database
Open Tools ->NuGet Package Manager->Package Manager Console
Run:
Update-Database
Identity tables are created automatically
Step 5: Run Application
Step 6: Register User by clicking Register link and entering Email and Password





Step 7: Login by clicking Login link and entering credentials



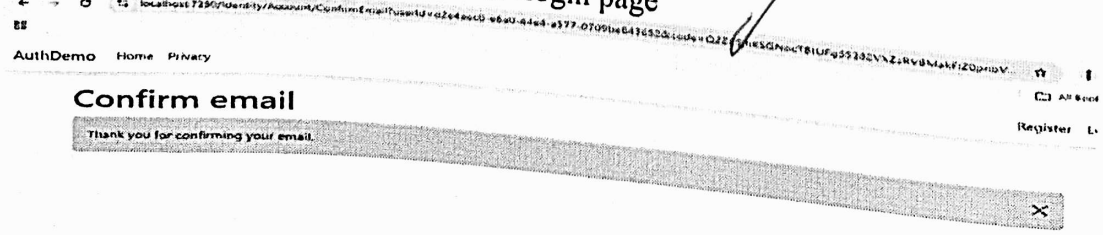
Step 8: Add Authorized Page.
Create new method in HomeController and apply Authorize attribute
using Microsoft.AspNetCore.Authorization;
Add new Method

```
[Authorize]
public IActionResult Secret()
{
return View();
}
```

Step 9: Create Secret View in Views/Home folder
Create folder only if needed.
Right-click on Home Folder → Add → View → Select Razor View Empty
Name: Secret.cshtml
Write in empty area below the prewritten lines:

```
<h2>Authorized Users Only</h2>
```

Step 10: Test Authorization by accessing /Home/Secret URL.
If logged in page opens else redirects to Login page



8/9/1

5D: Develop a web application to perform CRUD Operations(Create, Read, Update, Delete) on Employee table using EF Core.

1. Open Visual Studio 2026 and create a new project

2: Create MVC Project.

Select ASP.NET Core Web App Model-View-Controller

3: Configure Project. Set Project Name and Framework

Project Name :EmployeeCRUD and Framework .NET 10

4: Install EF Core Packages using NuGet Package Manager. Install

Goto: Tools->NuGet Package Manager->Manage NuGet Packages for Solution

Install:

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore

Microsoft.EntityFrameworkCore.Tools

Microsoft.EntityFrameworkCore.Design

Microsoft.VisualStudio.Web.CodeGeneration.Design

Use Same Version:10.08(or lower but same version for all above packages)

5. Create Employee Model

Right-click:

Models->Add->Class

Name:

Employee.cs

Write code:

```
Namespace EmployeeCRUD.Models
{ public class Employee {
public int Id { get; set; }
public required string Name { get; set; }
public required  string Department { get; set; }
public double Salary { get; set; } }}
```

4: Create Data Folder

Right-click project->Add->New Folder->Name:Data

5: Create DbContext Class

Inside Data Folder:

Add-> Class

Name:AppDbContext

AppDbContext.cs

```
using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;
namespace EmployeeCRUD.Data {
public class AppDbContext : DbContext {
public AppDbContext(DbContextOptions<AppDbContext> options)
: base(options){}
public DbSet<Employee> Employees { get; set; } }}
```

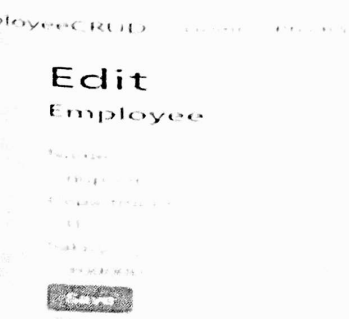
8/9/26

6: Add Connection String in appsettings.json file for EmployeeDB

```
"ConnectionStrings": {
"MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustSer
verCertificate=True"
```

7: Configure DbContext in Program.cs file using AddDbContext and UseSqlServer

using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
Before:
Var app=builder.Bulid();
Add:
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(
builder.Configuration.GetConnectionString("MyConnection")));
8. Create Employee Controller using Scaffold.
Right-Click:Controllers
Select:
Add->New Scaffolded Item
Choose:
Select MVC Controller with views, using Entity Framework
Click Add
9: Configure Scaffold by selecting Model class as Employee and Data context class as AppDbContext .Click on Add
10: Create Database
Open Tools ->NuGet Package Manager->Package Manager Console
Run: Add-Migration InitialCreate
Note** If the Migration shows Some errors or serious Warning then:
1. Open the File
Locate the ProductEFCore.csproj file inside your C:\Users\Administrator\source\repos\ProductEFCore\directory. Open the file with Notepad or any good text editor.
Paste the Code FiX before </Project>tag.
<ItemGroup>
<NuGetAuditSuppress Include="https://github.com/advisories/GHSA-g4vj-cjjj-v7hg" />
</ItemGroup>
And Bulid and rebuild the project again and run Migration Command again
Once Success
Update-Database
Step 11: Run Application
Open URL and ADD append following in front of url
/Employees





Practical 6A

Create a simple website for a college Information Portal and convert it into a progressive web application.

1.Create ASP.NET Core Web App

Search: `ASP.NET Core Web App (Model-View-Controller) → Click: Next

Project Name: `CollegePortalPWA`

Framework: `NET 10 or Latest Available` → Click: Create

2.Update Home Page

`Views → Home → Index.cshtml`:Replace existing Html code with

```
<div class="text-center">
<h1>Vartak College Information Portal</h1>
<h3>Welcome to Vartak College</h3>
<p>Courses Offered:</p>
<ul>
<li>BSC IT</li>
<li>BSC CS</li>
<li>BSC DS</li>
</ul>
<p>Contact: 9818763456</p>
</div>
```

3.Add PWA Support - Service Worker

Right-click `wwwroot` → `Add → New Item` → `JavaScript File`

Name: `service-worker.js`

```
self.addEventListener('install', function (event) {
console.log('Service Worker Installed');
});
self.addEventListener('fetch', function (event) {
console.log('Fetching:', event.request.url);
});
```

4.Create Manifest File

Right-click `wwwroot` → `Add → New Item` → `JSON File` →

Name: `manifest.json`

```
{
 "name": " Vartak College Information Portal",
 "short_name": "CollegePortal",
 "start_url": "/",
 "display": "standalone",
 "background_color": "white",
 "theme_color": "blue",
 "icons": [
  {
   "src": "/icons/icon1.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
   "src": "/icons/icon2.png",
   "sizes": "512x512",
   "type": "image/png"
  } ]}
```

5. Create Icons Folder
: Right-click `wwwroot` → `Add → New Folder` → Name: `icons` Add:
`icon1.png`
`icon2.png
6. Register Manifest
: `Views → Shared → _Layout.cshtml`
Add manifest link tag inside `<head>` section.
<link rel="manifest" href="~/manifest.json">
7.Register Service Worker
: Same `_Layout.cshtml` file, before `</body>`
<script>
if ('serviceWorker' in navigator)
{ navigator.serviceWorker.register('/service-worker.js')
.then((registration) => {
console.log('Service Worker Registered!', registration);
}).catch((error) => { console.log('Service Worker Registration Failed:', error);
});}</script>
8.Verify Static Files Enabled
`Program.cs`
Ensure this line exists, if not then add at appropriate place:
app.UseStaticFiles();
9.Run Application
Run in Chrome. Click on PWA Install Icon in address bar to install the app.

collegeportalpwa Home Privacy

Vartak College Information Portal

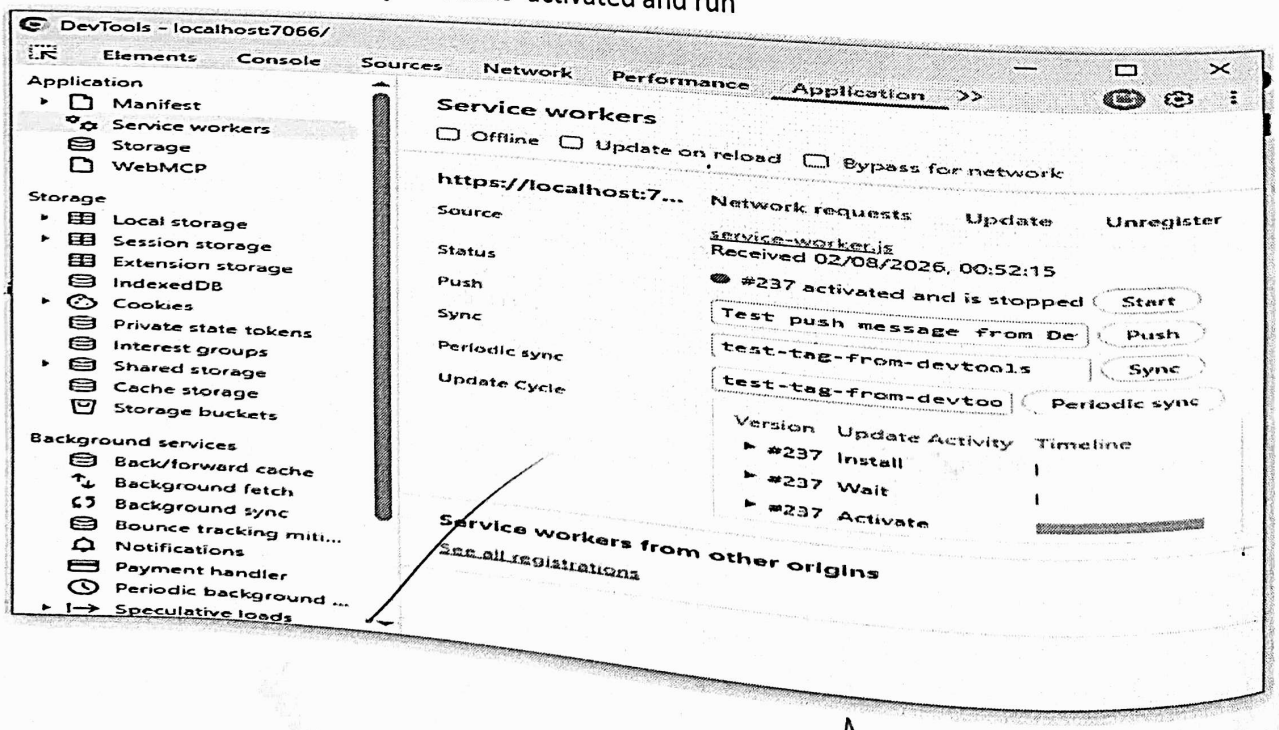
Welcome to Vartak College

Courses Offered:

- BSC IT
- BSC CS
- BSC DS

Contact: 9818763456

10.Verification of Service Worker Press `F12` → Go to `Application → Service Workers`
Check if `service-worker.js` status is `activated and run



create a PWA that display a custom offline page when user is not connected to the internet.

Practical 6B

1. Create ASP.NET Core MVC Projec:Project Name: `OfflinePWA`:
2. Framework: `.NET 10 or Latest Available` → Click: Create
3. Create Home Page:Views → Home → Index.cshtml`

```
<div class="text-center">
<h1> VARTAK College Information Portal</h1>
<h3>Welcome to VARTAK College</h3>
<p>This website supports Offline Mode.</p>
</div>
```

4. Create Offline Page
5. : Right-click `wwwroot` → `Add → New Item → HTML Page` →Name: `offline.html`."

```
<!DOCTYPE html>
<html>
<head>
<title>Offline</title>
</head>
<body>
<h1>No Internet Connection</h1>
<h3>Please check your internet connection.</h3>
</body></html>
```

6. Create Manifest File: `wwwroot` → `Add → New Item → JSON File`
7. →Name: `manifest.json`

```
{
"name": "Offline PWA",
"short_name": "OfflinePWA",
"start_url": "/",
"display": "standalone",
"background_color": "White", "theme_color": "blue",
"icons": [
{
"src": "/icons/icon1.png", "sizes": "192x192", "type": "image/png"
},
{
"src": "/icons/icon2.png", "sizes": "512x512", "type": "image/png"
}]}
```

Create Icons Folder: `wwwroot` → `Add → New Folder` → Name: `icons`

Add two icon files inside it.

- icon1.png
- icon2.png

8. Create Service Worker for Offline Caching: `wwwroot` → `Add → New Item → JavaScript File` →Name: `service-worker.js`

```
const CACHE_NAME = "offline-cache-v1";
const urlsToCache = [ "/","/offline.html"];
self.addEventListener("install", event => {
event.waitUntil(caches.open(CACHE_NAME).then(cache => {
return cache.addAll(urlsToCache);
}));});self.addEventListener("fetch", event => { event.respondWith(
fetch(event.request)
```

.catch(() => { return caches.match("/offline.html");}));});

9. Register Manifest:Views → Shared → _Layout.cshtml` → Add link tag in `<head>`

   <link rel="manifest" href="~/manifest.json">

10. Register Service Worker: Same `_Layout.cshtml` file, before `</body>

    <script>

if ('serviceWorker' in navigator) { navigator.serviceWorker.register('/service-worker.js')

    .then(() => console.log('Service Worker Registered'));

    }

    </script>

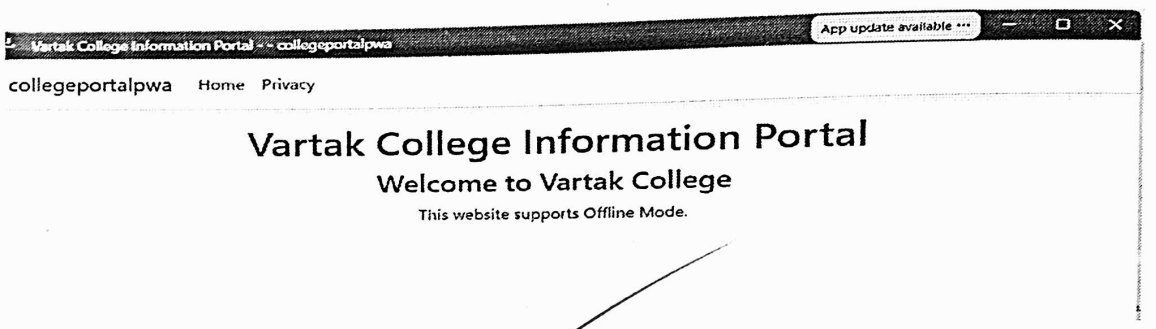
11. Verify Static Files: `Program.cs` →Add

    `app.UseStaticFiles();`

12. Run Application

13. **Verify Service Worker**

    Press `F12` → `Application → Service Workers` → Verify status.



14. Test Offline Page

    Press `F12` → `Network` tab → Select `Offline` → Refresh page.

28/9/26

Practical 7a

Develop a PWA that works in offline mode using service Worker.

1.Create ASP.NET Core MVC Project

Open Visual Studio 2026 → Create new project → Select `ASP.NET Core Web App (Model-View-Controller)

Project Name: `SWOfflinePWA` → Framework: `.NET 10` → Click: Create

2.Create Home Page

: `Views → Home → Index.cshtml` Replace existing code with

```
<div class="text-center">
<h1>College Information Portal</h1>
<h3>PWA Offline Demo</h3>
<p>This application works without internet connection. </p>
</div>
```

3.Create Manifest File

: Right-click `wwwroot` → `Add → New Item` → `JSON File` Name: `manifest.json`

```
{
"name": "Offline PWA",
"short_name":"OfflinePWA",
"start_url": "/",
"display": "standalone",
"background_color":"white",
"theme_color": "blue",
"icons": [
{
"src":"/icons/icon1.png",
"sizes": "192x192",
"type": "image/png"
},
{
"src":"/icons/icon2.png",
"sizes": "512x512",
"type": "image/png"
}]}
```

4.Create Icons Folder

: Right-click `wwwroot` → `Add → New Folder` → Name: `icons` Add: `icon1.png` 192x192 `icon2.png` 512x512

5.Create Service Worker

: Right-click `wwwroot` → `Add → New Item` → `JavaScript File` Name: `service-worker.js`

```
const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = [ "/","/Home/Index"];
self.addEventListener("install", event =>
{ event.waitUntil(caches.open(CACHE_NAME)
.then(cache => {
return cache.addAll(urlsToCache);
}));});
self.addEventListener("fetch", event => { event.respondWith(
caches.match(event.request)
.then(response => {
return response || fetch(event.request);
}));});
```

6.Register Manifest

: `Views → Shared → _Layout.cshtml` → inside `<head>`

add link tag for `manifest.json`

<link rel="manifest" href="/manifest.json">

7.Register Service Worker

: Same `_Layout.cshtml` file, before `</body>` tag

<script>

if ('serviceWorker' in navigator)

{ navigator.serviceWorker.register('/service-worker.js')

.then(() => console.log('Service Worker Registered'));

}</script>

Verify Static Files

: `Program.cs` → Ensure `app.UseStaticFiles();` exists

7.Run Application

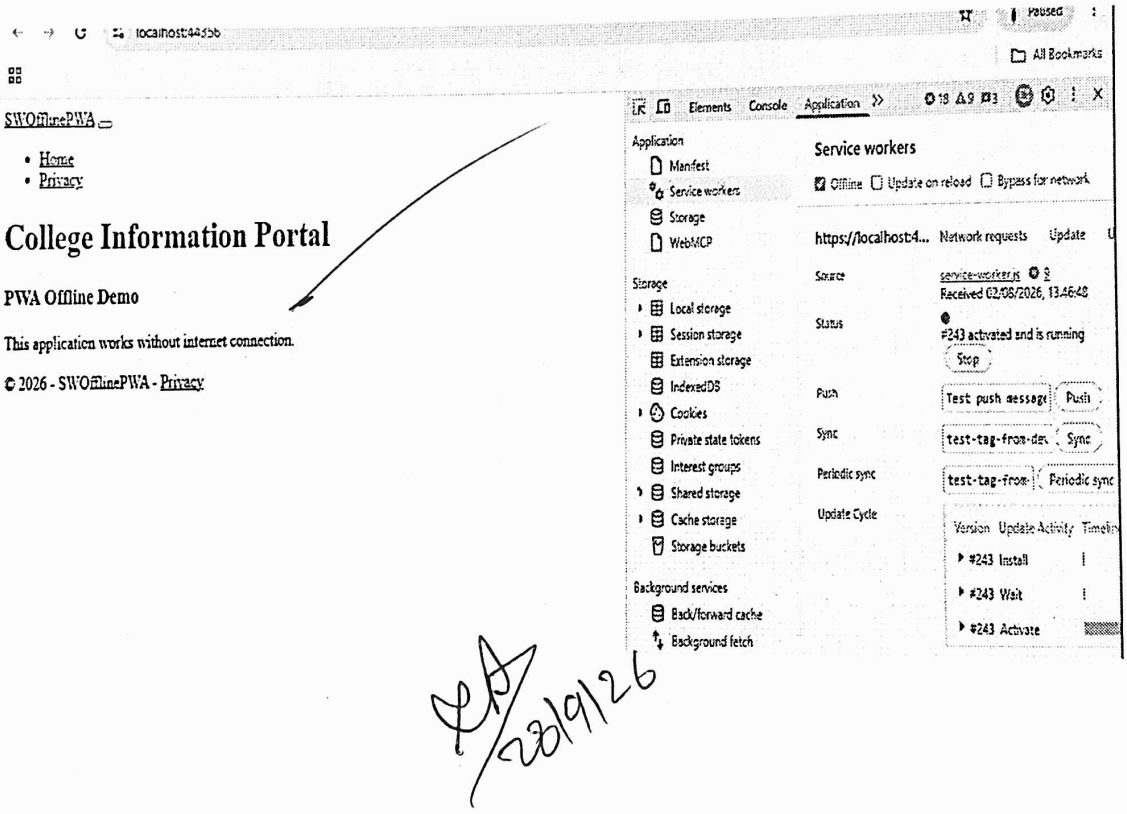
Run project and check PWA Install icon in browser

Verify Service Worker

Press `F12` → `Application → Service Workers` → Check `service-worker.js` status= `activated and running`

Test Offline Mode

Press `F12` → `Network` tab → Select `Offline` → Refresh webpage



28/9/26

Practical 8A

Create a PWA Application with an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json .

- **Create Empty Web Project** Create new project: Select `ASP.NET Core Web App (Model-View-Controller) Click: Next
- Configure Project

  Project Name: `BasicPWA`Framework: `.NET 10 or Latest` Click: Create
- Enable Static Files
- Open `Program.cs
- app.UseDefaultFiles();
- app.UseStaticFiles();

app.Run();

- Create Starting HTML Page

  : Right-click `wwwroot` → `Add New Item` → `HTML Page` Name: `index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Basic PWA</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="manifest" href="manifest.json">
</head>
<body>
    <h1>My Progressive Web Application</h1>
    <p>This is the starting page of PWA.</p>
</body>
</html>
```

- Link Manifest File in HTML

  Inside `<head>` tag of `index.html` add link to `manifest.json`

  <link rel=”manifest” href=”manifest.json”>

  Create Icons Folder:Right-click `wwwroot` → `Add → New Folder` → Name: `icons`

  Add: icon1.png
  icon2.png

- Create Manifest File

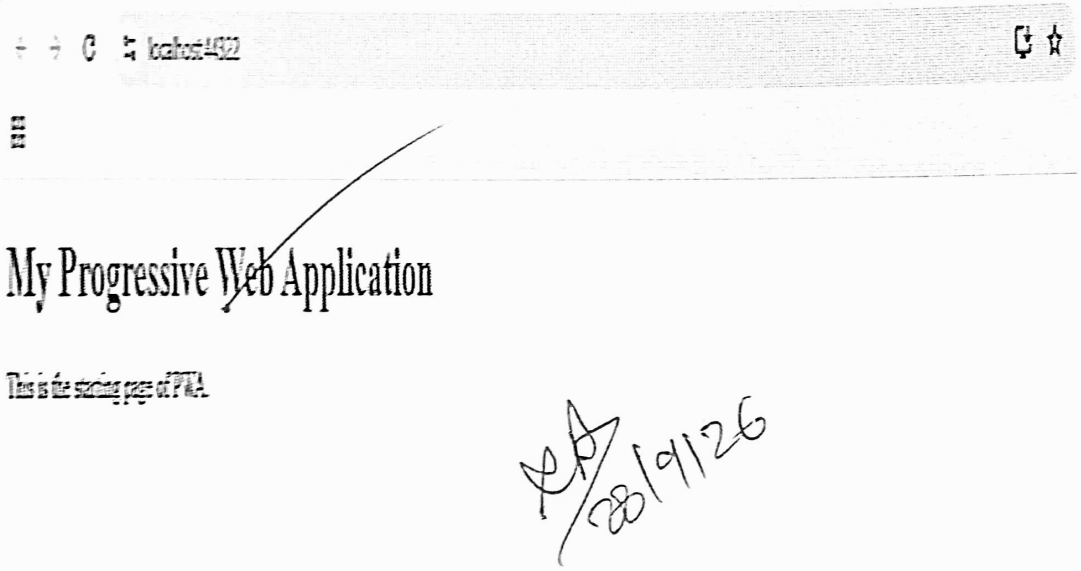
  : Right-click `wwwroot` → `Add → New Item` → `JSON File` Name: `manifest.json`

```
{
  "name": "Basic Progressive Web Application",
  "short_name": "BasicPWA",
  "description": "Simple   Example",
```

```
"start_url":"/index.html",
"display":"standalon",
"background_color":"white",
"theme_color":"blue",
"orientation":"portrait",
"icons": [
  {
    "src":"/icons/icon1.png", "sizes":"192x192","type": "image/png"
  },
  {
  "src": "/icons/icon2.png", "sizes": "512x512", "type": "image/png"}]
```

Run Application

Browser opens: `https://localhost:xxxx/index.html` PWA Install button appears in address bar



28/9/26

8(B) Create a simple Web API that returns student data in JSON format.

Algorithm / Steps:

1. Create Web API Project

Open Visual Studio 2026

Create new project

Select `ASP.NET Core Web API`

Click: Next

2. Configure Project

Project Name: `StudentAPI`

Framework: `.NET 10 or Latest`

Click: Create

3. Create Student Model

: Right-click Project → `Add → New Folder`

Name: `Models`

Inside Models: `Add → Class` → Name: `Student.cs`

```
namespace StudentAPI.Models {
public class Student {
public int Id { get; set; }
public string Name { get; set; }
public string Course { get; set; }
public int Age { get; set; }
}}
```

4. Create API Controller

: Right-click Project → `Add → New Folder` → Name: `Controllers`

Inside Controllers: `Add → Controller` → Choose: `API Controller - Empty`

Name: `StudentController.cs`

5. Write API code in the File

: `StudentController.cs`

Add routing attribute and create method to return list of student objects

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;
namespace StudentAPI.Controllers
{
[ApiController]
[Route("api/[controller]")]
public class StudentController : ControllerBase {
[HttpGet]
public List<Student> GetStudents() {
return new List<Student>() {
new Student
{
Id = 1,
Name = "Rahul",
Course = "BSc IT",
Age = 20
},
new Student
{
Id = 2,
Name = "Sneha",
Course = "BCA",
Age = 21
} };}}}
```

6. Run Application

Run the project

7. Test API

Open browser and go to: `https://localhost:xxxx/api/student`

Output: Student data is displayed in JSON format

9 A Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Steps:

1. Create ASP.NET Core Web API Project

Open Visual Studio 2026 → Create New Project

Select `ASP.NET Core Web API`

Project Name: `FactorialAPI`

Framework: `.NET 8` → Enable OpenAPI Support → Click: Create

2. Create Controller

: `Controllers` folder → Add Class: `MathController.cs`

3. Write Factorial Logic

Create GET method with route `factorial/{number}`

Check if number is negative, return BadRequest

Use loop to calculate factorial of the given number

Return result as JSON with Number and Factorial

```
using Microsoft.AspNetCore.Mvc;
namespace FactorialAPI.Controllers {
[Route("api/[controller]")]
[ApiController]
public class MathController : ControllerBase {
[HttpGet("factorial/{number}")]
public IActionResult GetFactorial(int number) {
if (number < 0){

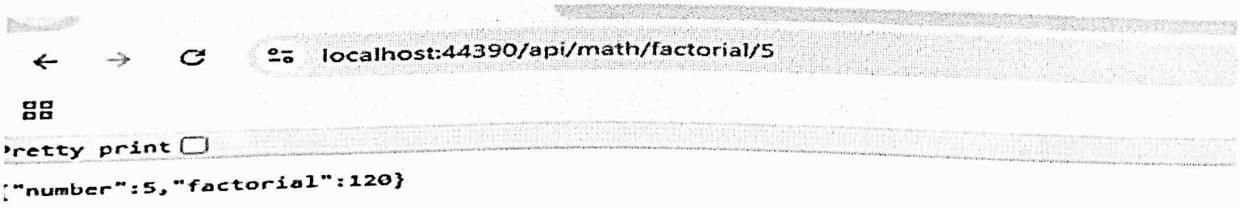
return BadRequest("Factorial not possible for negative numbers."); }
long factorial = 1;
for (int i = 1; i <= number; i++) {
factorial = i; }
return Ok(new {
Number = number,
Factorial = factorial
}); }}}
```

4. Run the Project

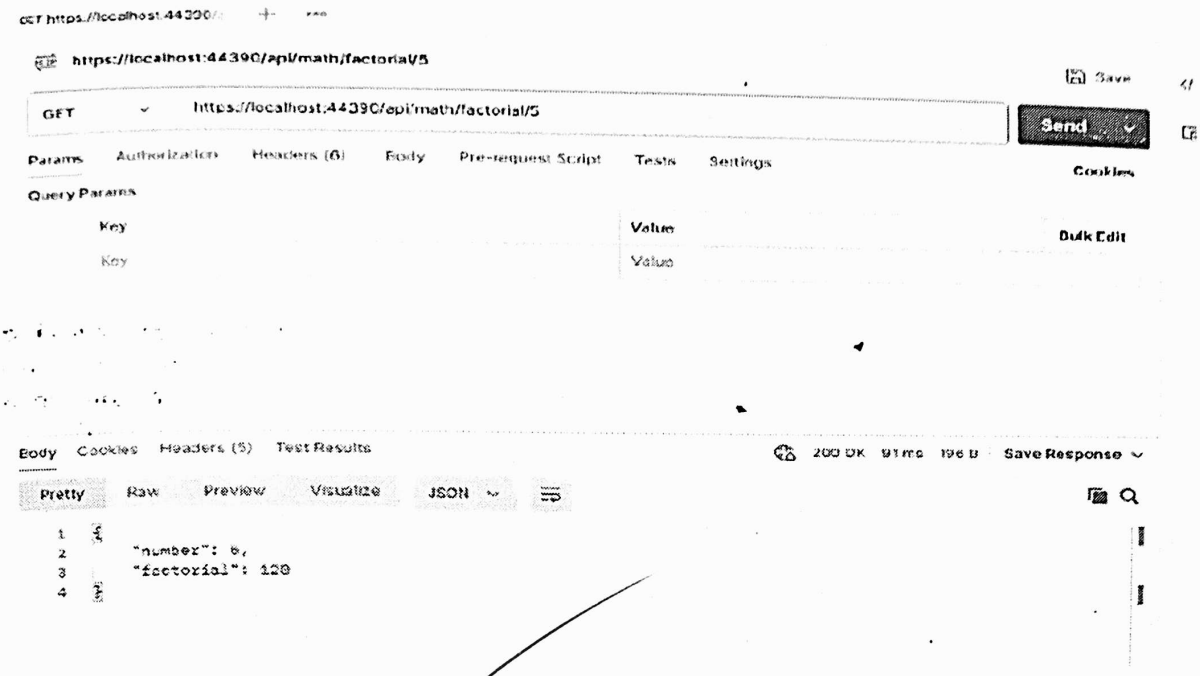
Project will run at `https://localhost:xxxx`

5. Test API in Browser

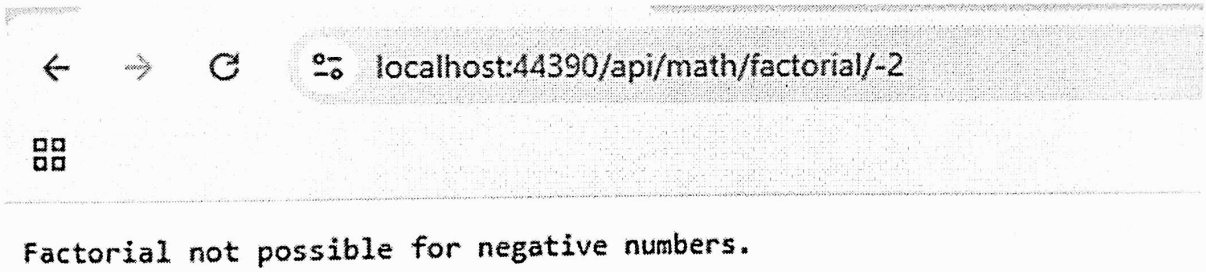
Open URL: https://localhost:44335/api/math/factorial/5



6. Test using Postman
Open Postman → Select Method: `GET`
Enter URL: `https://localhost:xxxx/api/math/factorial/5`
Click: Send



7. Test Negative Number
Enter URL: `https://localhost:xxxx/api/math/factorial/-2`
Click: Send



PRACTICAL-10
(A) Create a PWA with a web App Manifest file including:
 App name
 Icons
 Start URL
 Display mode
 Implement Push Notifications in a PWA.
Steps:
1. Create Project
Open Visual Studio 2026 → Create new project
Select `ASP.NET Core Empty Web App`
Project Name: `PushNotification`
Click: Create
2. Create Manifest File
: Right-click Project → `Add → New Folder` → Name: `wwwroot`
Inside `wwwroot` → `Add → New Item` → `JSON File`
Name: `manifest.json`

```
{
"name": "My PWA Application",
"short_name": "MyPWA",
```

```
"start_url": "/index.html",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#0d6efd",
"icons": [
{
"src": "icons/icon1.png",
"sizes": "192x192",
"type": "image/png"
},
{
"src": "icons/icon2.png",
"sizes": "512x512",

"type": "image/png" }]}
```

3. Add Icons

Right-click `wwwroot` → `Add → New Folder` → Name: `icons`
Add: `icon1.png` 192x192
`icon2.png` 512x512

4. Create HTML Page

: `wwwroot` → `Add → New Item` → `HTML Page` → Name: `index.html`
Link `manifest.json` in `<head>` and `app.js` in `<body>`

```
<!DOCTYPE html>
<html>
<head>
<title>My PWA</title>
<link rel="manifest" href="manifest.json">
<meta name="theme-color" content="#0d6efd">
</head>
<body>
<h1>My Progressive Web App</h1>
<button onclick="showNotification()">
Show Notification
</button>
<script src="app.js"></script>
</body>
</html>
```

5. Create JavaScript File for Notification

: `wwwroot` → `Add → New Item` → `JavaScript File` → Name: `app.js`

```
if ('serviceWorker' in navigator) {
navigator.serviceWorker.register('service-worker.js')
.then(() => console.log('Service Worker Registered')); }
function showNotification() {

Notification.requestPermission().then(permission => {
if (permission === 'granted') {
navigator.serviceWorker.ready.then(registration => {
registration.showNotification('PWA Notification', {
body: 'Push Notification Working Successfully!',
icon: 'icons/icon1.png'
}); }); }); }
```

6. Create Service Worker

`wwwroot` → `Add → New Item` → `JavaScript File`
Name: `service-worker.js`

```
self.addEventListener('install', event => {
console.log('Service Worker Installed');
});
self.addEventListener('activate', event => {
console.log('Service Worker Activated');
});
```

7. Configure Program.cs`Program.cs`

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();
app.UseDefaultFiles();
app.UseStaticFiles();
app.Run();
```

8. Run Application

Browser opens at `https://localhost:7236`
PWA Install icon appears in address bar. Click to install

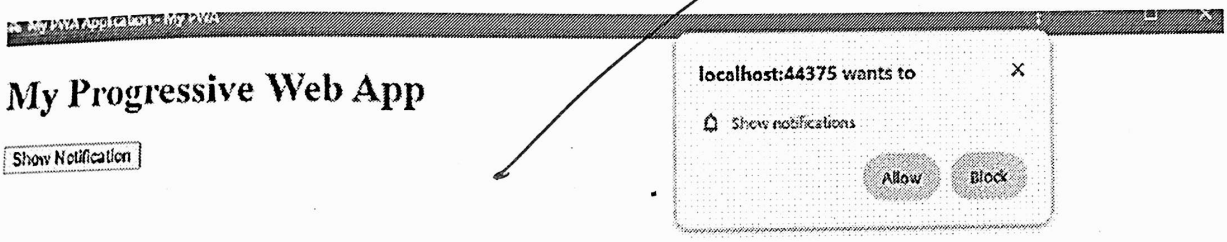
9. Install PWA

Open website in Chrome → Click Install icon → App installs like mobile app



10. Test Push Notification

Click "Show Notification" button → Browser asks permission → Click Allow → Notification appears



21/10/26